

三木书斋数据库线程化重构

从 DatabaseManager + BookQueryWorker 到统一 DatabaseWorker

一份面向学习和实施的完整方案：解释为什么要把耗时数据库操作放入工作线程，如何设计 Qt 信号槽、SQLite 连接、事务、查询结果和界面刷新，以及如何分阶段落地。

重要范围说明： 本文是设计与学习指南。本次只生成说明文档，不直接修改项目中的 `database.py`、`el_gui.py` 或其他业务代码。

适用项目： Eternity-Library（三木书斋）

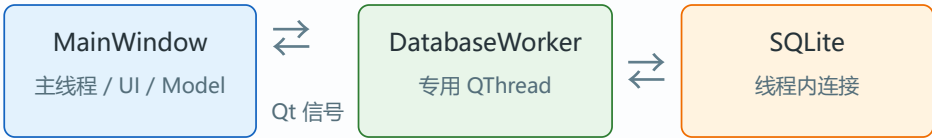
技术背景： Python 3.12+、PySide6、SQLite3

核心目标： 主线程只负责 UI；数据库读写在统一工作线程中串行执行；结果通过信号返回。

1. 先给结论：推荐怎样重构

推荐方案： 使用一个长期运行的 `DatabaseWorker` 和一个专用 `QThread`，让它负责所有数据库操作（查询、插入、更新、删除、事务和连接关闭）。`MainWindow` 不直接执行 SQL，也不持有工作线程的 SQLite 连接。

图 1：推荐的总体结构



这里的“统一”不是说所有代码必须写在一个大类里，而是指数据库连接和数据库操作的执行入口统一到一个工作线程。界面层只发出“我要查询”“我要添加”的请求，不直接调用 SQL。

为什么不优先拆成读线程和写线程

考虑点	两个线程（读/写分离）	一个 DatabaseWorker
SQLite 写入	仍然只有一个写者更安全，读写同步复杂	操作天然串行，事务顺序清晰
数据一致性	写入后何时刷新、查询是否读到旧数据，需要额外协调	写入完成后发出变更信号，再查询即可
学习成本	需要理解多个队列、连接和竞态	更容易理解和调试
当前项目规模	容易过度设计	足够支撑当前功能

2. 当前代码的主要问题

现有设计已经迈出了重要一步：`BookQueryWorker` 在查询函数内部创建独立 SQLite 连接，并在查询结束后关闭，这比把主线程连接直接传入工作线程安全得多。但仍有以下边界问题。

- 1. **写入仍在主线程：**`ScanBookFiles.write_into_database()` 循环调用 `DatabaseManager.add_book()`，每本书还可能执行多次 SQL 和提交事务，大量文件会卡住 UI。
- 2. **连接职责分裂：**一个对象维护长期连接，另一个对象每次查询创建连接；读写后的刷新和错误处理容易出现不一致。
- 3. **查询结果错误处理不完整：**当前查询函数把异常文本和正常列表放在同一个返回类型中，调用方可能误把错误字符串当成书籍列表。
- 4. **筛选查询没有回传信号：**按类型、状态、作者查询后没有发出 `books_ready`，因此 UI 无法可靠刷新。
- 5. **一次性加载全部字段：**`SELECT *` 会把所有列、所有记录一次性传入主线程；主线程更新 Model 和 View 仍可能耗时。
- 6. **模型数据格式尚未统一：**数据库返回元组，而封面模型期望字典；这不是线程问题，但重构时应一起明确数据结构。

关键认识：“查询放后台”只解决 SQL 执行阶段的阻塞。结果回到主线程后，数据转换、Model 重置、视图布局和图片加载仍可能卡顿。因此线程化、分页、减少字段和渐进式渲染要配合使用。

3. Qt 线程模型：必须先理解的概念

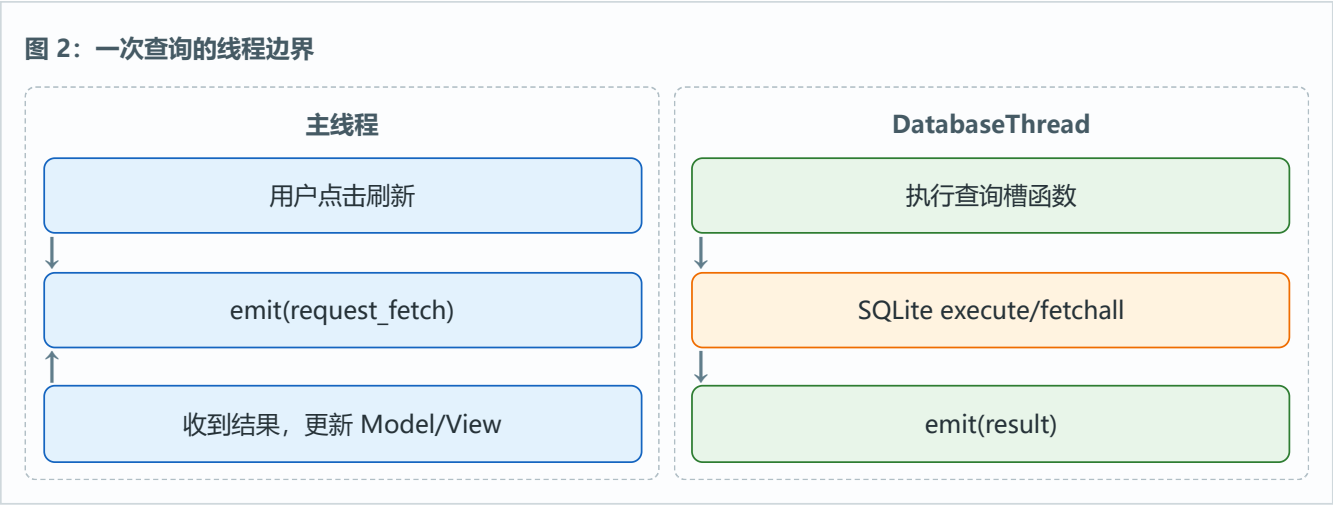
3.1 QThread 与 Worker 的关系

`QThread` 是线程控制对象；`DatabaseWorker` 是实际执行任务的对象。常见模式是：主线程创建 Worker，把 Worker 用 `moveToThread()` 移交给工作线程，再通过信号触发 Worker 的槽函数。

```
主线程创建 QThread
主线程创建 DatabaseWorker
DatabaseWorker.moveToThread(QThread)
连接 request_* 信号 → Worker 槽函数
连接 Worker 结果信号 → MainWindow 槽函数
QThread.start()
发出 request_* 信号，工作线程排队执行槽函数
```

3.2 为什么不能在工作线程直接操作 Qt Model 和 Widget

Qt 的大多数 GUI 对象，包括 `QTableView`、`QListView`、`QAbstractTableModel` 的数据变更，都应在主线程完成。工作线程只返回普通 Python 数据（例如列表、元组或数据类）。主线程收到信号后，再调用 `beginResetModel()`、`endResetModel()` 等方法。



4. 推荐的职责划分

对象	负责什么	不负责什么
MainWindow	按钮、状态栏、加载提示、接收结果、更新 View	不执行 SQL，不持有 Worker 的连接
DatabaseWorker	连接、建表、查询、增删改、事务、错误信号	不访问 QWidget/QTableView，不弹 QMessageBox
BookTableModel / CoverCardModel	保存主线程数据并通知 View 绘制	不查询数据库，不创建线程
ScanBookFiles	扫描文件、整理待入库数据	不直接 new 数据库连接；通过请求信号提交
数据库辅助函数	构造参数化 SQL、转换记录、定义字段常量	不依赖 UI 对象

5. DatabaseWorker 的接口设计

建议先设计清楚信号和槽，再开始改代码。接口不必一次覆盖所有功能，先实现“查询全部 + 批量写入 + 删除 + 关闭”即可。

5.1 请求信号与结果信号

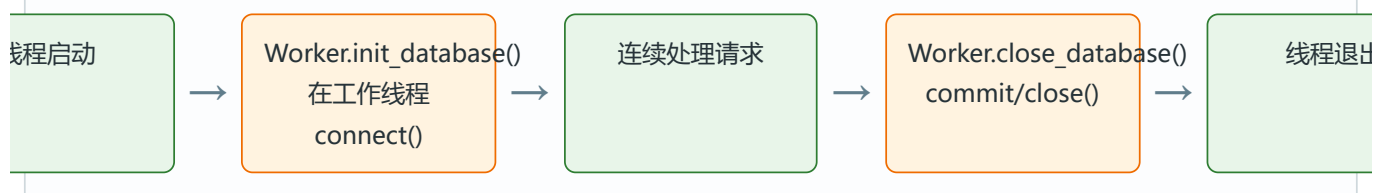
```
class DatabaseWorker(QObject):
    # MainWindow → Worker
    request_fetch_all = Signal()
    request_fetch_filtered = Signal(str, object) # filter_name, filter_value
    request_insert_many = Signal(list)
    request_delete = Signal(int)
    request_update = Signal(int, dict)

    # Worker → MainWindow
    books_ready = Signal(list)
    operation_succeeded = Signal(str)
    operation_failed = Signal(str)
    database_ready = Signal()
    database_closed = Signal()
```

信号参数应尽量表达“业务结果”，不要把 `sqlite3.Cursor`、连接对象或 `Worker` 本身传到主线程。错误统一走 `operation_failed`，主线程再决定显示状态栏、弹窗还是记录日志。

5.2 Worker 内部连接的生命周期

图 3：数据库连接生命周期

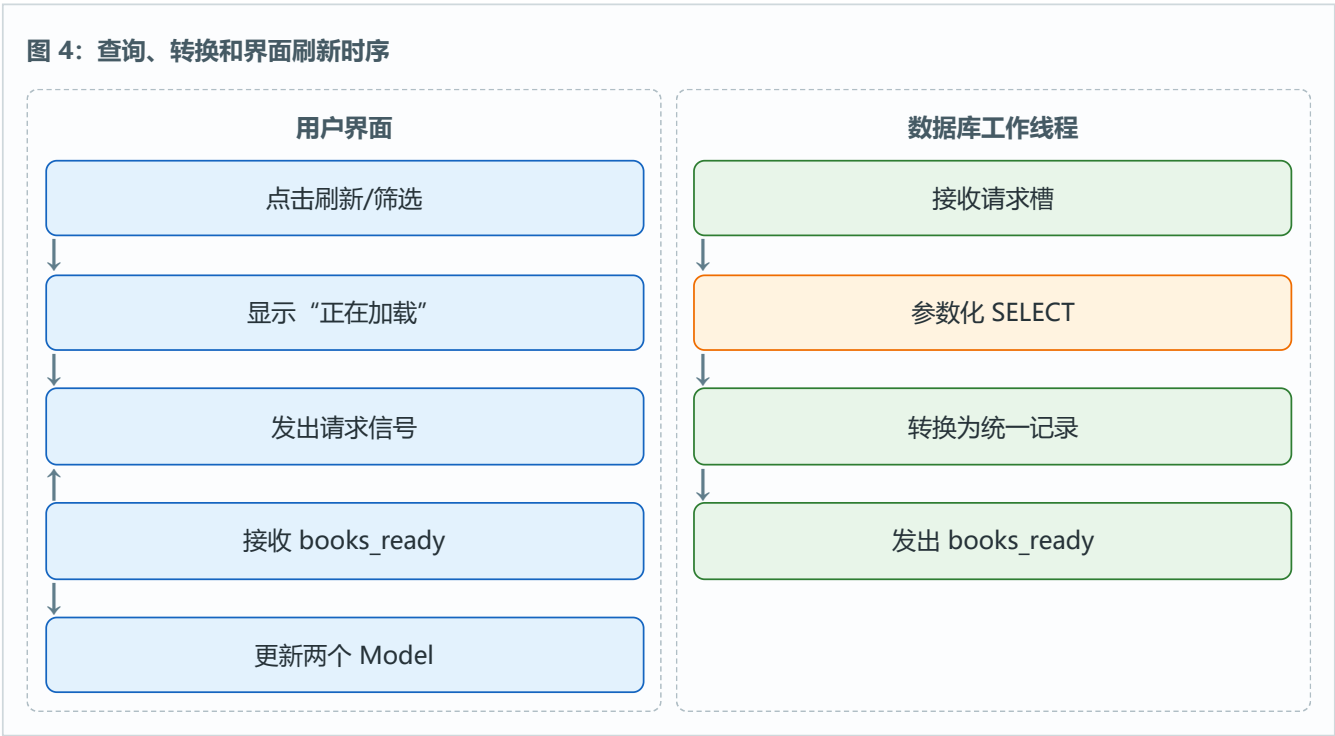


连接有两种可选策略：

- **长期连接（推荐当前项目）**：线程启动时创建，所有操作使用同一个连接，关闭线程时关闭。代码简单、事务连续。
- **每次请求新建连接**：每个槽函数内部 `connect()`、执行、关闭。隔离性更强，但重复创建连接、事务和初始化逻辑更多。

无论选择哪种策略，都必须保证“连接在哪个线程创建，就在哪个线程使用和关闭”。不要在主线程创建连接后，再把连接对象传入工作线程。

6. 读操作的完整流程



6.1 查询语句的建议

```
-- 封面模式只取必要字段
SELECT book_id, book_name, author, book_type, book_path
FROM books_information
ORDER BY book_id
LIMIT ? OFFSET ?;

-- 列表模式按需取完整字段
SELECT book_id, book_name, book_path, add_time, author, nationality,
       translator, publisher, publication_date, level, read_status,
       book_type, isbn, pages, read_progress, read_time, read_date,
       read_link, introduction
FROM books_information
ORDER BY book_id;
```

不要把用户输入直接拼接到 SQL 中。值使用 `?` 参数绑定；列名和排序字段如果必须动态选择，应使用代码中的白名单映射，而不是直接接收任意字符串。

6.2 空结果、错误和过期请求

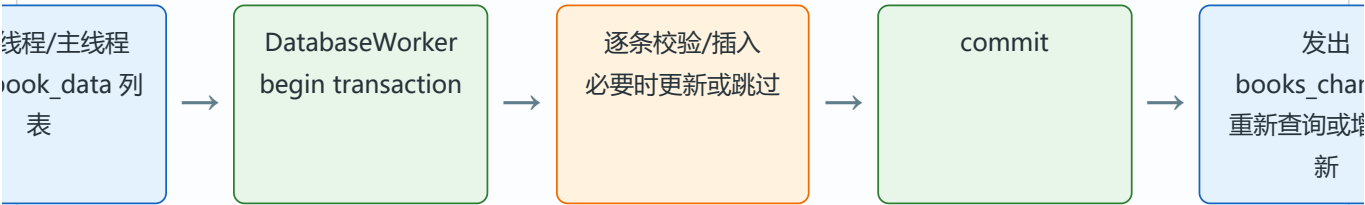
情况	Worker 行为	主线程行为
查询成功且有数据	发出 <code>books_ready(list)</code>	替换 Model 数据
查询成功但为空	仍发出空列表	清空 Model，并显示“暂无书籍”
SQL 失败	回滚（如有事务），发出错误信号	恢复按钮状态，提示用户
连续快速筛选	为请求附带 <code>request_id</code> 或筛选条件	只接受最新请求，避免旧结果覆盖新结果

7. 写操作与事务设计

7.1 为什么批量写入必须移动到工作线程

扫描目录后，通常会一次产生多本书。当前实现每次插入后都可能提交一次事务；如果有重复判断、生成 ID、更新 ID 等步骤，SQL 次数会更多。主线程在这些操作全部完成之前无法处理界面事件。

图 5：批量入库流程



7.2 事务的基本规则

```
try:
    connection.execute("BEGIN")
    for book in books:
        insert_or_handle_duplicate(book)
    connection.commit()
except sqlite3.Error:
    connection.rollback()
    raise
```

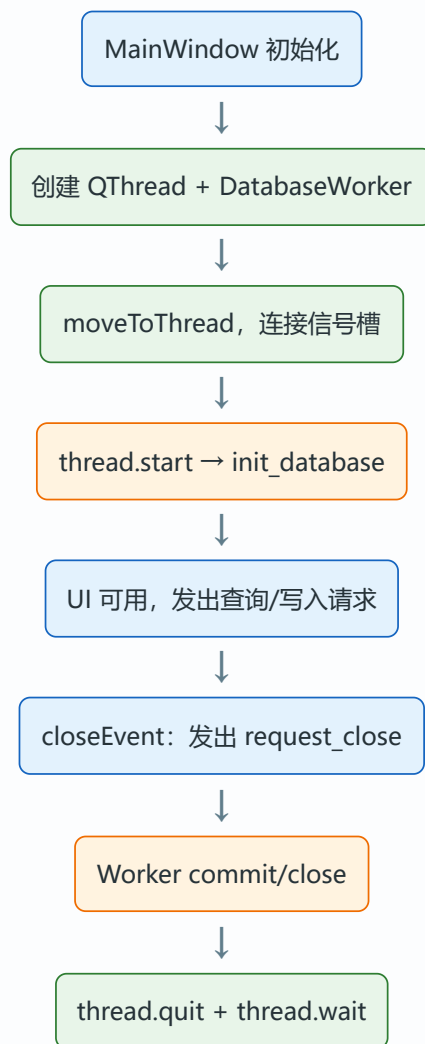
- 一批书籍要么整体成功，要么整体回滚；如果业务要求“部分成功”，应逐条记录结果，但仍要明确哪些记录成功。
- 不要在事务中弹 UI 对话框。重复书籍的策略应在请求中提前指定，或 Worker 发出“发现重复”信号后暂停当前任务，由主线程返回用户选择。
- 更简单的第一阶段方案是：扫描时先检查重复并采用固定策略（跳过），后续再实现可交互的覆盖/重命名。

7.3 重复书籍交互的两种方案

方案	流程	适合阶段
A：预先询问	主线程先查询重复项 → 弹窗 → 用户选择 → 把最终数据和策略发送给 Worker	第一版，流程简单
B：Worker 暂停请求	Worker 发现重复 → 发出 duplicate_found → 主线程询问 → 主线程发回 decision → Worker 继续	需要复杂交互时

8. 主线程与 Worker 的初始化/关闭流程

图 6：应用生命周期



关闭顺序很重要：不要先销毁 Worker 或数据库连接，再让线程继续运行；也不要还有写入请求时直接强制终止线程。关闭窗口时应停止接受新请求，等待当前任务完成，再关闭连接和线程。

9. 分阶段实施路线（不要求一次改完）

1. **阶段 0：统一数据结构。**明确数据库列顺序，定义字段常量或数据类，解决“元组/字典混用”问题。
2. **阶段 1：只迁移读取。**保留现有写入逻辑，建立统一 Worker，完成全部查询、筛选、空结果和错误信号。
3. **阶段 2：迁移批量写入。**把扫描结果作为列表发送给 Worker，在一个事务中处理；成功后发出变更信号。
4. **阶段 3：迁移删除和更新。**所有修改都走 Worker，禁止 UI 直接调用连接或游标。
5. **阶段 4：性能优化。**增加分页、字段裁剪、索引和缓存；必要时对封面生成、图片读取单独做任务。
6. **阶段 5：清理旧接口。**确认没有调用方后，删除旧的主线程连接、重复的 Worker 实例和无效的同步查询方法。

每个阶段都要验证什么

- 启动应用后数据库文件和表能正常创建。

- 空数据库可以正常显示空状态，而不是保留旧数据。
- 查询期间窗口仍能移动、点击和关闭。
- 批量扫描时界面不会长时间卡死。
- 插入失败时事务会回滚，并且用户能看到错误。
- 写入完成后列表和封面视图都能刷新。
- 快速连续筛选不会出现旧结果覆盖新结果。
- 关闭窗口时没有 “QThread destroyed while thread is still running” 警告。

10. 性能优化的优先级

优先级	措施	解决的问题
1	所有读写放入 Worker	避免 SQL 阻塞主线程
2	批量写入使用一个事务	减少每条记录 commit 的开销
3	查询只取需要的字段	减少 SQLite、Python 和信号传输数据量
4	封面模式分页/分批加载	避免一次重绘数千个卡片
5	为筛选列建立索引	加快类型、状态、作者筛选
6	缓存稳定查询结果	减少重复查询，但要在写入后失效

```
CREATE INDEX IF NOT EXISTS idx_books_book_type
ON books_information(book_type);

CREATE INDEX IF NOT EXISTS idx_books_read_status
ON books_information(read_status);

CREATE INDEX IF NOT EXISTS idx_books_author
ON books_information(author);
```

索引不是越多越好：它会加快查询，但会增加写入成本和数据库体积。应根据实际筛选频率决定。

11. 推荐的伪代码骨架

以下只是帮助理解结构的伪代码，不是要求直接复制到项目中的完整实现。

```
class DatabaseWorker(QObject):
    books_ready = Signal(list)
    operation_failed = Signal(str)
    books_changed = Signal()

    @Slot()
    def initialize(self):
        self.connection = sqlite3.connect(self.db_name)
        self.create_table()

    @Slot()
    def fetch_all(self):
        try:
            rows = self.connection.execute(
                "SELECT book_id, book_name, author, book_type "
                "FROM books_information ORDER BY book_id"
            ).fetchall()
        except sqlite3.Error as error:
            self.operation_failed.emit(str(error))
            return
        self.books_ready.emit(rows)

    @Slot(list)
    def insert_many(self, books):
        try:
            with self.connection:
                for book in books:
                    self.insert_one(book)
        except sqlite3.Error as error:
            self.operation_failed.emit(str(error))
            return
        self.books_changed.emit()

    @Slot()
    def close_database(self):
        if self.connection is not None:
            self.connection.close()
        self.database_closed.emit()
```

12. 最终建议清单

建议做

- 采用一个专用 DatabaseWorker 线程。
- 连接在 Worker 所在线程创建和关闭。
- 所有数据库操作通过信号槽请求。
- 主线程只更新 Model 和 View。
- 批量写入使用事务。
- 空结果也发正常结果信号。
- 逐步加入分页、索引和字段裁剪。

不要做

- 不要把 SQLite connection 跨线程传递。
- 不要在 Worker 中操作 QWidget。
- 不要让查询错误伪装成正常列表。
- 不要每本书都单独 commit。
- 不要用 SELECT * 支撑所有视图。
- 不要为了“读写分离”过早创建多个数据库线程。
- 不要在关闭窗口时强制杀死仍在写入的线程。

结语

这次重构的核心不是把代码“搬进线程”这么简单，而是建立清晰的边界：**UI 提出请求，DatabaseWorker 执行数据库操作，结果通过信号返回，Model 在主线程更新**。对于当前项目，单个数据库工作线程比读写双线程更容易学习、验证和维护；等数据规模和功能复杂度真正增长后，再考虑分页加载、任务队列或更细粒度的并发优化。

Eternity-Library guide · 数据库线程化重构学习指南 · 本文仅新增指南文件，不修改项目业务代码